

Documento Técnico, Modelo de Propensión de Compra + Agente IA

Caso Ferreycorp · Abril 2026

1. Objetivos

1.1 Objetivo de negocio

Desarrollar un sistema que prediga la propensión de compra de un cliente en una visita dada, y permita al equipo comercial **explorar y filtrar** las predicciones en lenguaje natural, sin depender de tableros estáticos ni de equipos de datos para responder preguntas específicas.

1.2 Objetivos técnicos

- Construir un modelo de machine learning con AUC-ROC mayor o igual a 0.65 y un **lift @ top-10% mayor o igual a 2.0**.
- Garantizar **explicabilidad** de las predicciones a nivel global (SHAP) y por cliente.
- Ofrecer una **arquitectura de acceso rápido** (menos de 100ms por consulta) para el agente.
- Diseñar la solución **portable entre nubes** (DigitalOcean, AWS, GCP) sin reescribir código.

2. Arquitectura de la solución

2.1 Componentes

Capa	Tecnología	Función
Modelo	LightGBM + Optuna	Predicción binaria de compra por (cliente, día)
Explicabilidad	SHAP	Interpretación global y por instancia
Storage	Parquet en S3-compatible	Tabla de predicciones, leída por el agente
Engine analítico	DuckDB embebido	Consultas SQL sub-segundo sobre Parquet (no requiere servidor)
Agente IA	Claude Sonnet + tool use	Traduce preguntas en filtros estructurados
UI	Streamlit	Chat, dashboard y métricas del modelo

Container	Docker	Portabilidad cloud-agnóstica
-----------	--------	------------------------------

2.2 Diseño cloud-agnóstico

La arquitectura se ejecuta sin cambios en las tres principales nubes. La portabilidad se logra evitando servicios propietarios y favoreciendo estándares abiertos:

Componente	DigitalOcean	AWS	GCP
Object storage	Spaces (S3-compatible)	S3	GCS (vía HMAC, equivalente a S3)
Compute	App Platform	ECS Fargate o App Runner	Cloud Run
LLM	Anthropic API directa	Anthropic API directa	Anthropic API directa
Cache (opcional)	Managed Redis	ElastiCache	Memorystore
BD relacional (opcional)	Managed Postgres	RDS	Cloud SQL

El switch de proveedor implica únicamente cambiar variables en `.env`, el código aplicativo es idéntico.

2.3 Tool-use estructurado vs. Text-to-SQL

El agente **no escribe SQL libre**. En lugar de eso, expone tres funciones estructuradas:

- `query_predictions(filters, order_by, limit, columns)`
- `aggregate_predictions(group_by, aggregations, filters)`
- `schema_info()`

El LLM emite filtros en formato JSON. Estos se validan contra una **whitelist de columnas y operadores** y se traducen a SQL parametrizado en DuckDB. Esto:

- Elimina el riesgo de SQL injection (en cualquier modalidad).
- Hace la solución **predecible y testeable**.
- Es casi tan flexible como text-to-SQL para los casos de uso comerciales.

3. Metodología

3.1 Definición del problema

- Granularidad:** una fila igual a una visita de un cliente en un día específico.
- Target:** `incidencia_compra`, valores en {0, 1}.
- Tarea:** clasificación binaria.
- Volumen:** 58,693 visitas, 500 clientes únicos, 730 días (aprox. 2 años).
- Distribución del target:** 24.94% de positivos (desbalance moderado).

3.2 Pipeline

CSV crudo ■ ■■ EDA (11 figuras y reporte en docs/eda/REPORTE_EDA.md) ■ ■■ Feature Eng. (43 features causales y segmentación K-Means; ■ sin leakage: solo info de días anteriores) ■ ■■ Train (Split temporal train/val/test = 70/15/15) ■ 1. LogReg baseline ■ 2. LightGBM + Optuna (30 trials, optimiza AUC val) ■ ■■ Explain (SHAP, lift curve, calibration, decile table) ■ ■■ Score (Predicciones por cliente y día en Parquet, consultable por el agente vía DuckDB)

3.3 Feature engineering causal

Para evitar **data leakage**, todas las features históricas se calculan usando únicamente los datos con `dia_visita < día actual` del mismo cliente:

Familia	Features	Descripción
RFM	prior_visits, prior_purchases, prior_buy_rate, prior_qty, prior_avg_qty_per_purchase, recency_days	Recencia, frecuencia, monto histórico
Lealtad	loyalty_b{1..5}, loyal_brand_id	% histórico de compras por marca; marca dominante
Sensibilidad a promo	buy_rate_with_promo, buy_rate_no_promo, promo_uplift	Cómo responde el cliente a promos históricamente
Precio relativo	rel_price_b{1..5}, min_rel_price_today, loyal_brand_rel_price	Precio actual vs. promedio histórico
Estado del día	any_promo_today, pct_brands_on_promo, precio_marca_{1..5}, promo_marca_{1..5}	Variables de la visita actual
Demografía	edad, ingreso_anual, genero, estado_civil, nivel_educacion, ocupacion	Estáticas por cliente
Contexto	loyal_brand_on_promo, ultima_marca_comprada, ultima_cantidad_comprada	Interacciones

Validación de no-leakage: en la primera visita de cada cliente, todas las features `prior_*` valen 0 y `recency_days` = -1 (sentinel). Validado programáticamente.

3.4 Segmentación de clientes

K-Means (k=4) sobre features agregadas a nivel cliente: `buy_rate`, `total_purchases`, `avg_qty_per_purchase`, `edad`, `ingreso_anual`. Etiquetas heurísticas:

Segmento	n	Buy rate real	Interpretación
Leales premium	47	59.6%	Alta frecuencia e ingreso alto
Cazadores de oferta	45	28.7%	Frecuencia media, sensibles a precio
Compradores moderados	139	22.0%	Comportamiento promedio
Visitantes ocasionales	269	17.7%	Baja frecuencia

Nota: la segmentación se computa sobre todo el periodo y se usa solo como feature para que el agente filtre (no como input del modelo, para no introducir leakage).

3.5 Validación temporal

Crítica para evitar leakage en la evaluación:

Split	Días	Filas
Train	1 a 509	41,061
Validación	510 a 619	8,433
Test	620 a 730	9,199

3.6 Modelos entrenados

Baseline, Logistic Regression: - `class_weight=balanced` para mitigar desbalance. - `StandardScaler` en numéricas, categóricas como `int`.

LightGBM (modelo final): - Boosting GBDT, manejo nativo de categóricas. - Tuning con Optuna (30 trials, dirección igual a maximizar AUC val). - Early stopping con paciencia de 50 rondas. - Hiperparámetros optimizados: `num_leaves`, `max_depth`, `learning_rate`, `feature_fraction`, `bagging_fraction`, `min_child_samples`, `lambda_l1`, `lambda_l2`.

4. Resultados

4.1 Métricas de modelo (test set)

Métrica	LogReg	LightGBM
AUC-ROC	0.673	0.684

PR-AUC	0.458	0.457
Log loss	0.572	0.521
Brier score	0.190	0.170
Lift @ top 10%	2.36	2.40
Lift @ top 20%	1.87	1.90
Lift @ top 30%	1.66	1.63

4.2 Análisis de deciles

Decil	n	Score promedio	Tasa real	Lift vs. base
D1 (top 10%)	5,870	0.500	69.6%	2.40 veces
D2	5,869	0.334	42.7%	1.40 veces
D3	5,869	0.279	31.7%	1.10 veces
D4	5,869	0.246	24.7%	1.07 veces
D5	5,869	0.226	20.1%	0.90 veces
D10 (bottom 10%)	5,870	0.162	7.0%	0.55 veces

Lectura: enviando una campaña al top 10% scoreado, **70% de ellos compran**, frente al 25% en la población general (2.4 veces la tasa esperada). Si se extiende al top 20%, se captura aprox. 38% del total de compras llamando solo al 20% de los clientes.

4.3 Calibración

El modelo está bien calibrado tras Optuna: las probabilidades predichas por decil (0.50 en D1, 0.16 en D10) son consistentes con las tasas reales (70% en D1, 7% en D10). Brier score igual a 0.170 vs. 0.187 baseline trivial (predecir prior).

4.4 Explicabilidad (SHAP)

Top features por impacto promedio: 1. `prior_buy_rate` (correlación con target igual a +0.38) 2. `prior_purchases` (+0.28) 3. `recency_days` (-0.16: más reciente, más probable) 4. `min_rel_price_today` (-0.06: precios más bajos hoy, más probable) 5. `any_promo_today` (+0.06) 6. `loyal_brand_on_promo`, `loyal_brand_rel_price` (interacciones cliente y día)

Insight de negocio: la mayor señal predictiva proviene del comportamiento histórico del cliente; las features demográficas aportan poco. Esto valida el enfoque de propensión por visita (vs. por cliente estático).

5. Agente conversacional

5.1 Diseño

Usuario: "Dame los 20 clientes top de Leales premium con score mayor a 0.5" ■ ▼ Claude Sonnet vía Anthropic API interpreta y emite tool call: { "name": "query_predictions", "input": { "filters": [{"column": "cluster_label", "operator": "=", "value": "Leales premium"}, {"column": "score_compra", "operator": ">", "value": 0.5}], "order_by": [{"column": "score_compra", "direction": "desc"}], "limit": 20 } } ■ ▼ Validación whitelist, luego SQL parametrizado, luego DuckDB sobre Parquet (S3, Spaces o GCS) ■ ▼ Resultados de vuelta al LLM, que redacta una respuesta en lenguaje natural

5.2 Casos de uso soportados

- Listar top-K clientes según múltiples filtros (segmento, edad, ingreso, decil, marca leal).
- Agregaciones por grupo (score promedio por segmento, # clientes en decil top con promo).
- Validación del modelo (comparar score predicho vs. tasa real por subconjunto).
- Análisis de campañas (qué % de compras se capturan en el top-N).

6. Captura de valor en el negocio

6.1 Áreas impactadas

Área	Impacto	KPI
Marketing dirigido	Reducir costo de campañas contactando solo al top-K	Costo de campaña (menos 50% al cortar bottom-50%)
Pricing y promociones	Identificar clientes con alta sensibilidad a promo	Lift incremental por promo personalizada
Reactivación	Detectar clientes activos con baja propensión actual	Tasa de reactivación, # clientes recuperados
Operación comercial	Auto-servicio para análisis ad-hoc	Tiempo de respuesta a preguntas (de horas a segundos)
Pricing dinámico	Identificar elasticidad por segmento	Margen por unidad por segmento

6.2 Estimación de valor (top-down, conservadora)

Asunción: 500 clientes, aprox. 25% conversión histórica, ticket promedio aprox. 3 unidades a precio promedio S/2. Universo anual de aprox. 25,000 visitas con 6,250 compras por aprox. S/6 igual a aprox. S/37.5k de revenue baseline en 2 años.

Escenario campaña dirigida al top-20% por score (en lugar de campaña masiva al 100%): - Costo de contacto cae **80%**. - Captura de compras: aprox. **38%** del total (curva de lift). - Si el costo de contactar al 100% es S/X, el ahorro es 0.80 por S/X manteniendo 38% de la efectividad, lo que da una rentabilidad por contacto aprox. 5 veces.

Escenario activación de "Cazadores de oferta" con promociones personalizadas: - 45 clientes con frecuencia aprox. 25%, lo que produce aprox. 3.4 visitas por mes. - Si la promo personalizada incrementa buy rate 5pp absolutos (de 29% a 34%), se generan aprox. 7 compras adicionales por mes en este segmento.

6.3 KPIs propuestos

KPI	Frecuencia	Meta inicial
Lift de conversión campañas vs. control random	Mensual	mayor o igual a 2.0 veces
Costo por compra incremental (CAC marginal)	Mensual	menos 40% vs. baseline
Tasa de reactivación (clientes inactivos más de 30 días que vuelven a comprar)	Trimestral	más 10pp
Tiempo medio de resolución de queries comerciales (uso del agente)	Mensual	menor a 2 min
Calibración del modelo (Brier score en producción)	Mensual	menor o igual a 0.18
Drift de scores vs. baseline	Semanal	Alerta si la distribución de D1 cambia más de 15%

7. Cronograma propuesto

Semana	Tarea
1	EDA y diseño de features causales
2	Modelado (baseline + LightGBM + tuning)
3	Explicabilidad, métricas de negocio y agente IA
4	UI Streamlit, despliegue cloud y documentación

8. Próximos pasos y extensiones

- 1. **Modelo multiclase de marca:** predecir cuál marca elige el cliente cuando compra.
- 2. **Re-entrenamiento periódico** automatizado con Airflow, Prefect o Cloud Scheduler.
- 3. **A/B testing framework** para validar el lift en producción contra grupo control.
- 4. **Monitoreo de drift** con Evidently o Whylabs.
- 5. **Feature de engagement digital** si llega data de canal online.

6. **Pricing optimizer** sobre el output del modelo (ej: simular impacto de cambio de precio en propensión por segmento).